

George Eliot Archive

Fiction Audiobook Developer Manual

Authors:

Colin Patrick

Evan Ginos

Traveon Wright

Alex Nolin

Submitted in partial fulfillment of the requirements for COMP4710
Senior Design to the Department of Computer Science and Software
Engineering, Samuel Ginn College of Engineering, Auburn University

Auburn, Alabama

April 29th, 2025

Executive Summary:

This document provides a technical overview of the George Eliot Archive web application. The application serves as a digital platform for accessing George Eliot's literary works, offering integrated text display and synchronized audio playback (LibriVox and TTS). It operates as a client-side application, leveraging modern web technologies and external data hosting for efficiency and scalability. This manual outlines the system's architecture, structure, interfaces, operating requirements, dependencies, and core features to facilitate understanding, ongoing development, and maintenance.

Table of Contents:

1. Architecture
2. Structure
 - File Structure
 - HTML Structure
 - JavaScript Structure (script.js, widget.js)
 - Data Structure (bookData)
- 3.
4. Interfaces
 - User Interface (UI)
 - Code Interfaces (Key Functions)
 - External Interfaces
5. Assumptions
6. Dependencies
7. Features

1. Architecture

The George Eliot Archive is a client-side web application, functioning as a Single Page Application (SPA) for browsing and interacting with George Eliot's works. It relies entirely on **HTML, CSS, and JavaScript**

running in the user's web browser. There is no server-side application logic beyond serving the static files (HTML, CSS, JS) and the data files (XML, MP3).

Core Components:

- **Presentation Layer (HTML & CSS):** Defines the structure (index.html) and visual appearance (style.css, Google Fonts) of the archive, including the sidebar navigation, main content area, and the audio player widget.
- **Logic & Interaction Layer (JavaScript):** Handles all dynamic behaviour:
 - Fetching and parsing book data (TEI XML).
 - Dynamically updating the UI (sidebar navigation, breadcrumbs, main content display).
 - Managing application state (current book, chapter, audio source).
 - Controlling the HTML5 audio player.
 - Implementing UI features like the draggable/resizable audio widget and the collapsible sidebar.
- **Data Layer (External):** Textual content (TEI XML) and audio narrations (MP3 files) are hosted externally on GitHub repositories and fetched on demand using their raw content URLs.

Data Flow:

1. **Initial Load:** The browser loads index.html, which links to style.css, Google Fonts, widget.js, and script.js. The initial UI shows the main book index in the sidebar and a welcome message. The preferred audio source is loaded from localStorage.
2. **Book Selection:** User clicks a book link in the sidebar (loadBooks(index) or loadVeil(index)).
3. **XML Fetch & Parse:** JavaScript fetches the corresponding TEI XML file for the selected book from the GeorgeEliotArchive/allworks GitHub repository via fetch (or XMLHttpRequest for loadVeil). The fetched XML text is parsed into a DOM object (xmlDoc).
4. **UI Update (TOC):** The parseXML function processes the xmlDoc's Table of Contents (div[type="toc"]). It updates the sidebar to show the book's structure (e.g., list of chapters, or Books containing chapters) and updates the main content area to list the chapters/books with corresponding download links. Breadcrumbs are updated.
5. **Chapter Selection:** User clicks a chapter link in the sidebar (loadChapterText(chapterId, element)).
6. **Text Display:** JavaScript finds the corresponding chapter content within the already parsed xmlDoc using XPath (//*[@xml:id="..."]). The chapter's heading and paragraphs (<p>) are extracted and displayed in the main content area.
7. **Audio Load:** The loadAudioForChapter function is called. It determines the correct audio filename based on the book (currentIndex), chapter (chapterId), and currently selected audio

source (currentAudioSource). It constructs the full URL pointing to the MP3 file in the GeorgeEliotArchive/AudioBooks repository (LibriVox or TTS).

8. **Audio Playback:** The constructed URL is set as the src for the HTML5 <audio> element in the widget. The widget UI (book title, chapter title, source dropdown) is updated, and the widget is displayed. The user can then interact with the audio controls.
9. **Audio Source Change:** User selects a different source from the dropdown (handleAudioSourceChange). The selection is saved to localStorage, and loadAudioForChapter is called again to load the audio from the new source.
10. **Download:** The User clicks a download button. downloadAudioFile constructs the appropriate URL and triggers a browser download.

2. Structure

File Structure

The application consists of the following core client-side files:

- **index.html:** The main HTML file defining the page structure.
- **style.css:** (Assumed) Contains all CSS rules for styling the application, including layout, typography, sidebar behaviour, and audio widget appearance.
- **widget.js:** Contains JavaScript code specifically related to the audio player widget's functionality (DOM references, audio controls, event listeners, drag-and-drop).
- **script.js:** Contains the main application logic (data definitions, state management, XML fetching/parsing, UI updates for sidebar/main content, audio loading logic tied to chapters).

External Data Dependencies (Hosted on GitHub):

- **Text:** TEI XML files located in https://github.com/GeorgeEliotArchive/allworks/tree/{TEXT_GITHUB_BRANCH}/teiEncode/. Accessed via raw.githubusercontent.com.
- **Audio:** MP3 audio files located in https://github.com/GeorgeEliotArchive/AudioBooks/tree/{AUDIO_GITHUB_BRANCH}/. Organized into LibriVox and TTS subdirectories, further subdivided by book (tts property in bookData). Accessed via raw.githubusercontent.com.

HTML Structure (index.html)

- Contains metadata (charset, viewport), the page title, links to Google Fonts (Crimson Text), and the external CSS file (style.css).
 - **(div):** The collapsible navigation sidebar.
 - Contains #breadcrumb for navigation context.
 - Initially populated with links to all works (<a> tags with onclick calling loadBooks or loadVeil). Dynamically updated to show book/chapter lists.
 -
 - **(div):** The main content area.
 - Contains .openbtn (button) to toggle the sidebar.
 - Contains #mainContent (div) where the welcome message, chapter listings (with download links), and chapter text are displayed dynamically.
 -
 - **(div):** The floating audio player widget.
 - .audio-widget-header: Draggable header containing title, chapter, drag handle (☰), and close button (×).
 - .audio-widget-content: Contains:
 - .audio-source-selector: Dropdown (#audioSource) to choose between LibriVox (A/B if available) and TTS.
 - #audioPlayer: The native HTML5 <audio> element.
 - .audio-controls: Custom buttons for Rewind, Play/Pause, Fast Forward.
 - .progress-container: Contains the seek bar (#seekBar).
 -
 -
 - **tags:** Link the external JavaScript files (widget.js, script.js) at the end of the body.
-

JavaScript Structure

The JavaScript logic is split conceptually (and potentially physically) into two main parts:

(Audio Widget Logic):

- **DOM References:** Variables holding references to elements within the #audioWidget.
- **State Variables:** isDragging, offsetX, offsetY for drag functionality.
- **Audio Control Functions:** togglePlayPause, updatePlayPauseButton, rewindAudio, forwardAudio, updateSeekBar, seekAudio, closeAudioWidget. These directly manipulate the #audioPlayer element and widget UI.

- **Event Listeners:** Attached to the audio player (play, pause, ended, timeupdate, etc.), seek bar (input), and widget header (mousedown for dragging).
- **Drag & Drop Logic:** onDragMouseMove, onDragMouseUp implement dragging.
- **Resizing:** Handled natively by CSS (resize: both), no specific JS for this.

(Main Application Logic):

- **Constants:** URLs for GitHub resources, repository details, LibriVox flags (LIBRIVOX_A, LIBRIVOX_AB).
- **Array:** The central data structure defining available books and their properties (see below).
- **Global State Variables:** currentIndex, currentChapter, currentChapterNo, xmlDoc (parsed XML), selectionType (TOC structure type), currentAudioSource (persisted in localStorage).
- **Helper Functions:** isDigit, extractNumber, nsResolver (for XPath).
- **UI Functions:**
 - toggleNav: Manages sidebar visibility and main content shifting.
 - loadBooks, loadVeil: Initiate loading of a work (fetch XML, call parser).
 - parseXML: Process loaded XML, populate sidebar TOC, and main content chapter list/downloads.
 - loadChapters: Display chapter list for a specific book/part.
 - loadChapterText: Display selected chapter text in #mainContent and trigger audio loading.
 - loadDownloads: Generate download buttons for a chapter.
 - loadAudioForChapter: Core logic to determine audio URL based on state, update widget UI, and set audioPlayer.src. Handles source availability and LibriVox A/B variants.
 - handleAudioSourceChange: Responds to dropdown changes, updates state, saves preference, and reloads audio.
 - downloadAudioFile: Constructs a download URL and triggers download.
 - goToIndex: Resets the application state and UI to the initial view.
-
- **Initial Setup:** DOMContentLoaded listener sets the initial audio source dropdown value.

Data Structure (bookData Array)

This array is central to the application's operation. Each object within the array represents a single work by George Eliot and has the following properties:

- name (String): The full title of the work (e.g., "Middlemarch").
- code (String): A short code used in audio filenames (e.g., "MM").

- **xml (String):** The URL-encoded filename of the TEI XML file in the allworks repository (e.g., 'Middlemarch_refine_v1.xml').
- **tts (String):** The URL-encoded directory name within the AudioBooks repository (under both LibriVox and TTS) containing the audio files for this work (e.g., 'Middlemarch').
- **vox (Boolean/null):** Indicates LibriVox audio availability and versioning:
 - null: No LibriVox audio available.
 - LIBRIVOX_A (false): A single version of LibriVox audio exists (files end in _CHAPTER{nn}.mp3).
 - LIBRIVOX_AB (true): Two versions (A and B) of LibriVox audio exist (files end in _CHAPTER{nn}A.mp3 and _CHAPTER{nn}B.mp3).
-

3. Interfaces

User Interface (UI)

- **Sidebar (**
 - Displays the initial list of works.
 - Updates to show the structure (Books/Chapters) of the selected work.
 - Allows selection of works and chapters via clickable links (<a>).
 - Contains breadcrumbs for navigation context.
 - Can be toggled open/closed via the → button (.openbtn).
-
- **Main Content Area (**
 - Displays a welcome message initially.
 - Displays the list of chapters (and download links) for a selected work/book.
 - Displays the full text of a selected chapter.
-
- **Audio Widget (**
 - Displays the current book title and chapter title.
 - Allows selection of audio source (LibriVox A/B, TTS) via dropdown (#audioSource).
 - Provides standard audio controls: Play/Pause, Rewind 10s, Fast Forward 10s, Seek bar.
 - Can be closed using the × button.
 - It can be dragged around the screen using its header.
 - Can be resized using the standard browser resize handle (bottom-right).
-

- **Download Links:** Buttons within #mainContent to download audio files directly.

Code Interfaces (Key Functions)

These functions serve as entry points for user interactions or internal workflows:

- `toggleNav()`: Called by the sidebar toggle button.
- `loadBooks(index)`: Called by clicking a work link in the initial sidebar index.
- `loadVeil(index)`: Called by clicking the "Lifted Veil" link.
- `parseXML(xmlString)`: Internal function called after fetching XML.
- `loadChapters(bookId)`: Called by clicking a "Book" link in the sidebar (if applicable) or internally by `parseXML`.
- `loadChapterText(chapterId, element)`: Called by clicking a chapter link in the sidebar.
- `loadAudioForChapter(chapter, setup)`: Called by `loadChapterText` and `handleAudioSourceChange`.
- `handleAudioSourceChange()`: Called when the #audioSource dropdown value changes.
- `downloadAudioFile(chapterId, tts, voxType)`: Called by clicking download buttons.
- `goToIndex()`: Called by clicking the "Index" breadcrumb.
- **Widget Functions** (`togglePlayPause`, `rewindAudio`, `forwardAudio`, `seekAudio`, `closeAudioWidget`): Called by interacting with the audio widget controls.

External Interfaces

- **GitHub Raw Content:**
 - Uses `fetch` (or `XMLHttpRequest`) to retrieve TEI XML files from <https://raw.githubusercontent.com/GeorgeEliotArchive/allworks/...>
 - Sets the `src` attribute of the `<audio>` element to MP3 URLs from <https://raw.githubusercontent.com/GeorgeEliotArchive/AudioBooks/...>
 - Generates download links pointing to the same MP3 URLs.
- **Google Fonts API:**
 - Fetches the 'Crimson Text' font via a `<link>` tag in the HTML `<head>`.
- **Browser APIs:**
 - **DOM API:** Extensively used for selecting, creating, modifying, and appending elements.
 - **Fetch API / XMLHttpRequest:** For network requests to GitHub.
 - **Web Storage API** (To store and retrieve the user's preferred audio source (`ge_audio_source`)).

- **HTML5 Audio API:** To control playback (play(), pause(), currentTime, duration, events).
- **DOMParser API:** To parse XML strings into DOM objects.
- **XPath API:** To query the parsed XML DOM (evaluate).

4. Assumptions

The code operates under the following assumptions:

- **Browser Compatibility:** The user's browser supports HTML5, CSS3, modern JavaScript (ES6+ including fetch, async/await, const/let), DOMParser, XPath, Web Storage (localStorage), and the HTML5 Audio element with MP3 playback.
- **File Availability:** index.html, style.css, widget.js, and script.js are served correctly.
- **GitHub Repository Structure:**
 - The GeorgeEliotArchive/allworks and GeorgeEliotArchive/AudioBooks repositories exist and are publicly accessible.
 - The specified branches (TEXT_GITHUB_BRANCH, AUDIO_GITHUB_BRANCH - currently 'main') exist.
 - The directory structure within these repositories matches the code's expectations (e.g., `teiEncode/`, `LibriVox/`, `TTS/`, book-specific subfolders named as per `bookData[i].tts`).
- **Data Consistency:**
 - The `bookData` array in `script.js` accurately reflects the available works, their codes, XML filenames, audio folder names, and LibriVox availability (`vox` flag).
 - XML files adhere to the expected TEI-like structure, particularly for TOC (`div[type="toc"] > list > ref` or `div[type="toc"] > list > item > ref`), chapter identification (`div[@xml:id]`), and content (`head`, `p`). The structure for "Lifted Veil" is handled slightly differently (`getElementsByTagName`).
 - Audio filenames strictly follow the pattern `{code}_CHAPTER{nn}[A/B].mp3`, where `{nn}` is a zero-padded two-digit chapter number.
- **Network Connectivity:** The user has an active internet connection to fetch XML and audio files from GitHub.
- **CSS Styling:** `style.css` provides all necessary styles for layout, visibility, the sidebar animation, and widget appearance/resizing/dragging cues.

5. Dependencies

- **External Libraries/Services:**

- Google Fonts ('Crimson Text').
- **External Data Sources (via HTTP):**
 - raw.githubusercontent.com for serving content from:
 - GeorgeEliotArchive/allworks repository (TEI XML files).
 - GeorgeEliotArchive/AudioBooks repository (MP3 audio files).
- **Internal Project Files:**
 - index.html
 - style.css (Required for visual presentation and layout).
 - widget.js (Required for audio widget functionality).
 - script.js (Required for core application logic).
- **Browser APIs:**
 - DOM API
 - Fetch API / XMLHttpRequest
 - Web Storage API (localStorage)
 - HTML5 Audio API
 - DOMParser API
 - XPath API

6. Features

- **Work Browsing:** Users can browse a list of George Eliot's works.
- **Table of Contents Navigation:** Supports works with simple chapter lists and works with hierarchical structures (e.g., Book > Chapter).
- **Text Reading:** Displays the formatted text content of selected chapters.
- **Audio Playback:**
 - Provides synchronized audio narration for chapters via an integrated audio player widget.
 - Supports multiple audio sources:
 - LibriVox (Community-read audio).
 - TTS (Text-to-Speech generated audio).
 - Handles multiple LibriVox versions (A/B) where available.
- **Audio Player Widget:**
 - Persistent: Remains visible across chapter navigation until closed.
 - Draggable: Can be moved around the screen.
 - Resizable: Can be resized by the user.
 - Standard Controls: Play/Pause, Rewind (10s), Fast Forward (10s), Seek bar.

- Informative: Displays current Book and Chapter titles.
 - Source Selection: Allows switching between available audio sources.
- **Audio Download:** Allows users to download MP3 files for individual chapters (LibriVox A/B and TTS, where available).
- **Collapsible Sidebar:** Provides more reading space by hiding the navigation menu.
- **Breadcrumb Navigation:** Shows the user's current location within the archive's structure (Index > Book > Chapter).
- **State Persistence:** Remembers the user's preferred audio source (LibriVox/TTS) across sessions using localStorage.
- **Specific Work Handling:** Includes special parsing logic for "The Lifted Veil" due to its different XML structure.

In conclusion, the George Eliot Archive application offers a robust and accessible platform for users to engage with these important literary works through integrated text and audio. The client-side architecture ensures efficient operation and leverages external data sources effectively. We are confident that this technical foundation provides a stable and maintainable system, well-positioned for future enhancements and continued service.